Optimizer and Learning-Rate Schedule Interactions for High-Dimensional Genomic Classification

ASDS 6304 — Optimization Methods · Group 5
University of Texas at Arlington · Spring 2026
Course Instructor: Prof. Ce Bian · May 3, 2026

Abstract

In machine-learning practice, the choice of optimization algorithm is more often dictated by convention than by evidence. For high-dimensional tabular problems such as cancer-type classification from gene expression data, practitioners typically default to Adam without comparing it against alternatives, and the rich literature on learning-rate schedules — cosine annealing, warm restarts, exponential decay — has been developed almost exclusively in the context of deep networks for vision and language. Whether those schedule families translate to shallow models on tabular biomedical data is an open question. We address both gaps with a structured factorial benchmark of six optimizers (full-batch GD, single-sample SGD, mini-batch SGD, SGD with momentum, Adam, and L-BFGS), four learning-rate schedules, and two regularizers (L1 and L2) on multinomial logistic regression for 33-class TCGA Pan-Cancer classification. The full grid produces 144 model fits over three random seeds. Working with a 495-sample stratified subset and 20,000 variance-filtered genes, we find that L-BFGS achieves the highest mean test accuracy (0.914), and that L-BFGS+L1 reaches the peak individual test accuracy of 0.919 deterministically across all 12 of its schedule × seed combinations. The schedule effect is small — 0.5 percentage points of total swing — substantially weaker than the optimizer effect, which spans nearly 10 percentage points. A linear-program formulation for clinical gene-budget selection identifies a 200-gene panel reaching test accuracy 0.929 with Lasso ranking, against 0.919 for the full 2,820-gene Lasso model. The takeaway: for shallow regularized classifiers on tabular high-dimensional data, the optimizer matters far more than the schedule, and classical second-order methods remain a strong baseline that adaptive methods should be required to beat.

1. Introduction

Cancer-type classification from bulk RNA-Seq is a canonical high-dimensional tabular machine-learning problem. The Cancer Genome Atlas provides expression measurements for roughly 60,000 transcripts across more than 10,000 tumour samples spanning 33 cancer types, a setting where feature dimensionality exceeds sample count by several orders of magnitude. The optimization landscape that emerges has well-known properties: the loss has many global minima with comparable training error, regularization is required to make the problem identifiable, and the choice of optimization algorithm controls not only the speed of convergence but also which solution among the equivalent minima is reached.

Despite the centrality of optimization in this regime, applied genomic studies rarely justify their algorithmic choice. Adam is used reflexively, often with default learning rate and no schedule; coordinate descent is used reflexively for Lasso. Crawford and Greene (2024) compared coordinate descent against stochastic gradient descent for Lasso multinomial logistic regression on TCGA and showed that the two algorithms select substantially different gene sets at convergence, despite reaching near-identical training and test loss. Their work, however, examined only two optimizers and did not study learning-rate schedules at all.

On the schedule side, the situation is the inverse. Schedules such as cosine decay (Loshchilov and Hutter, 2017), warm restarts, and exponential decay are standard ingredients in transformer and convolutional-network training, where they interact with batch normalization, attention, and very long training horizons. Their behaviour on shallow tabular models — no normalization layers, no architectural priors, short training budgets — is undocumented in the literature.

This project addresses both gaps. We run a single 6 × 4 × 2 factorial design across three random seeds, holding architecture, loss function, batch size, and training budget fixed so that any difference in performance is attributable to the optimization regime alone. The model is multinomial logistic regression, formulated in PyTorch as a single linear layer followed by a softmax. The 144 fits each produce per-epoch training loss, validation accuracy, final test accuracy, and wall-clock time. We additionally compare the resulting Lasso solutions against a linear-program formulation for gene-budget selection that operates independently of the optimizer.

Four research questions structure the work:

1. Among first-order methods (GD, SGD, Adam) and quasi-Newton methods (L-BFGS), which gives the best generalization on regularized logistic regression in 20,000-dimensional feature space?

2. Does learning-rate schedule choice matter as much as optimizer choice in this setting? Are some optimizers more schedule-sensitive than others?

3. How does the optimizer × schedule combination affect Lasso sparsity and the identity of the genes that survive regularization?

4. Given a clinical budget of k gene measurements, which subset maximizes class separability? Can a linear program produce a panel competitive with Lasso at the same cardinality?

2. Related Work

2.1 Optimization for high-dimensional classification

Multinomial logistic regression with elastic-net regularization has been the workhorse model for tumour-type classification from gene expression since at least the early 2010s. Friedman, Hastie, and Tibshirani (2010) established cyclic coordinate descent as the de-facto solver, and the glmnet package built around their algorithm became the standard reference implementation. Kingma and Ba (2015) introduced Adam, which combined per-parameter adaptive learning rates derived from running gradient moments with bias-corrected updates; the algorithm rapidly became the default for deep-network training and was adopted in computational biology by extension, although the theoretical case for adaptive methods on convex problems is weaker than for non-convex ones.

Liu and Nocedal (1989) developed the limited-memory BFGS (L-BFGS) algorithm as a quasi-Newton method that maintains an implicit Hessian approximation from a small history of gradient differences. L-BFGS does not scale to deep learning workloads — its line search and curvature memory require full-batch evaluations — but on smooth convex problems with moderate parameter counts it converges in a small number of iterations, which makes it well suited to logistic regression on tabular data. The Scikit-Learn LogisticRegression class uses L-BFGS as its default L2 solver for exactly this reason.

The most directly relevant prior work is Crawford and Greene (2024), which fit Lasso multinomial logistic regression on TCGA pan-cancer data using two optimizers — coordinate descent and SGD — and reported that the two solvers select substantially different gene subsets at convergence. Their finding motivates the present extension to a wider optimizer set and to combinations of optimizers with schedules.

2.2 Learning-rate schedules

The constant learning rate, where η is fixed throughout training, is the simplest schedule and serves as the universal baseline. Cosine annealing was introduced by Loshchilov and Hutter (2017) under the name SGDR and decays the learning rate from its initial value to near zero following a cosine curve, optionally restarting periodically. Cosine decay without restarts has since become the default schedule for large-scale image and language model training. Exponential decay, in which the learning rate is multiplied by a fixed factor $\gamma < 1$ at each epoch, is older and originates in the classical stochastic-approximation literature.

Empirical study of these schedules has been concentrated almost entirely on deep networks. Smith (2017) showed that cyclical learning rates can substantially improve convergence on convolutional architectures. To our knowledge, none of these schedule families has been systematically benchmarked on shallow tabular classifiers in a high-dimensional regime, where the gradient landscape is fundamentally different — there is no architectural depth, no batch normalization to interact with, and the training horizon is at most a few hundred epochs rather than tens of thousands of steps.

2.3 Feature selection as optimization

L1-regularized logistic regression performs implicit feature selection by driving redundant coefficients to exactly zero (Tibshirani, 1996). The geometry of the L1 ball — its corners on the coordinate axes — produces this sparsity in a way that L2 regularization cannot. In the gene-expression context, L1 selects one representative gene from each correlated module, which is biologically interpretable. The downside is that L1 is sensitive to the optimizer: different solvers reach different points on the boundary of the L1 ball, so the gene set that survives is not unique.

An orthogonal approach is to formulate gene selection as an explicit combinatorial optimization. If each gene *j* has a univariate discriminability score F_j — for example the Fisher F-statistic from one-way ANOVA — then choosing the k most informative genes can be cast as a linear program: maximize $\Sigma\, F_j\, x_j$ subject to $\Sigma\, x_j \le k$ and $0 \le x_j \le 1$. The constraint matrix is totally unimodular, so the LP relaxation is exact. We use the LP as a baseline against which to compare Lasso-selected gene sets at matched cardinality.

3. Methodology

3.1 Dataset and preprocessing

We use the TCGA Pan-Cancer expression matrix (RSEM, $\log_2(\text{count} + 1)$) from the UCSC Xena Toil Recompute hub, which harmonizes RNA-Seq across 33 cancer types using a single uniform pipeline. The full atlas contains 10,535 tumour samples and 58,581 HUGO-named genes after the Toil pipeline. Phenotype labels are the per-sample primary disease annotation provided in the Xena pancanatlas distribution; missing values are 0% by construction.

For the experiments reported here we work with a stratified subset of 495 samples — 15 patients per cancer type, drawn at random with seed 42. The subset preserves all 33 classes and keeps the sample count larger than the number of features that survive the variance filter, making the problem tractable on a single GPU within a few minutes per fit. The same code path supports the full 10,535-sample cohort by setting one Boolean flag; we leave that run for future work, since the small-cohort results are already saturated against the held-out test set.

Feature preprocessing is identical for every model. We remove duplicate gene columns, drop genes whose variance across the cohort is below 0.01 (which deletes 8,556 housekeeping or non-expressed transcripts: 58,581 $\rightarrow$ 50,025), and retain the top 20,000 most variable genes by sample variance. The samples are then split 60/20/20 into training, validation, and test sets with stratification by cancer type, yielding 297 / 99 / 99 samples respectively. A *StandardScaler* is fit on the training set only and applied to validation and test — essential because gradient-based optimizers are extremely sensitive to feature scale, and unscaled gene-expression data can vary across genes by factors of 10^5 or more.

3.2 Model architecture

Multinomial logistic regression is implemented in PyTorch as a single fully-connected layer of shape 20,000 $\rightarrow$ 33, with no hidden units, no nonlinearity except the implicit softmax in the loss, and no bias regularization. The loss is cross-entropy plus a regularization term $\lambda\|W\|_1$ or $\lambda\|W\|_2^2$ applied to the weight matrix only, with $\lambda = 10^{-4}$. Holding the architecture and loss fixed across the entire factorial means that any difference in test accuracy or convergence is attributable to the optimizer, the schedule, or the regularizer — never to architectural confounds.

3.3 Optimizers

Six optimizers cover the major families:

Optimizer	Update rule	Family
Full-batch GD	$w \leftarrow w - \eta \nabla L(w)$	Deterministic first-order
SGD (b=1)	$w \leftarrow w - \eta \nabla L_i(w)$	Stochastic first-order
Mini-batch SGD	$w \leftarrow w - \eta \nabla L_B(w)$, $\lvert B\rvert=32$	Stochastic first-order
SGD + Momentum	$v \leftarrow \beta v + \nabla L$; $w \leftarrow w - \eta v$, $\beta=0.9$	Stochastic w/ momentum
Adam	$m\blacksquare$, $v\blacksquare \rightarrow w \leftarrow w - \eta \cdot m\blacksquare/(\sqrt{v\blacksquare} + \varepsilon)$	Adaptive first-order
L-BFGS	$w \leftarrow w - \eta H^{-1} \nabla L$	Quasi-Newton

L-BFGS is run in full-batch mode with up to 20 inner iterations per outer step, as required by the algorithm's line-search dependence. Single-sample SGD uses batch size strictly equal to one (one update per training example, 297 updates per epoch); this is methodologically distinct from mini-batch SGD with $|B|=32$ and produces qualitatively different convergence behaviour.

3.4 Learning-rate schedules

Four schedules span the major families: **Constant** — $\eta_t = \eta_0$ throughout, the universal baseline; **Cosine decay** — $\eta_t = \eta_0 \cdot \tfrac{1}{2}(1 + \cos(\pi t/T))$, smoothly annealed from η_0 to zero over $T = 100$ epochs; **Exponential decay** — $\eta_t = \eta_0 \cdot \gamma^t$ with $\gamma = 0.97$, reaching roughly 5% of η_0 at the end of training; and **Warm restarts** — cosine schedule with period $T_0 = 25$ epochs, the learning rate periodically resetting to η_0.

Base learning rates are tuned per optimizer to keep all six in their stable regime: $\eta_0 = 0.01$ for GD, SGD, and Mini-batch SGD; 0.005 for SGD+Momentum; 0.001 for Adam; and 0.1 for L-BFGS. These values were selected from a preliminary log-spaced sweep on SGD (Section 5.6) and adjusted by family using standard heuristics: momentum allows a smaller base rate, Adam is less stable at $\eta_0 = 0.01$, and L-BFGS expects a larger initial step size because it scales it by an internal Hessian approximation.

3.5 Factorial design

The factorial is fully crossed: 6 optimizers × 4 schedules × 2 regularizers (L1, L2) × 3 random seeds = 144 model fits. Seeds are 42, 123, and 2026; the seed controls both PyTorch weight initialization and DataLoader shuffling. Each fit runs for exactly 100 epochs. We record per-epoch training loss, validation accuracy, learning-rate value, and at termination test-set accuracy, macro-F1, and wall-clock time.

3.6 Linear-program gene-budget extension

To address Research Question 4, we formulate gene selection under a clinical measurement budget as a linear program. For each gene *j* we compute a one-way-ANOVA Fisher F-statistic from the 33 cancer-type groups in the training set; this is a univariate measure of class-discriminability invariant to the choice of downstream model. We then solve, for each budget $k \in \{10, 25, 50, 100, 200\}$:

$$\text{maximize } \Sigma_j F_j x_j \text{ subject to } \Sigma_j x_j \le k,\ 0 \le x_j \le 1$$

via *scipy.optimize.linprog* with the HiGHS solver. The constraint matrix is totally unimodular and the LP relaxation returns integer-optimal solutions, so we recover a binary gene set without rounding. Each k-gene panel is evaluated on the held-out test set by fitting an unregularized multinomial logistic regression on those k features alone, which lets us compare LP panels against the top-k genes ranked by Lasso coefficient magnitude at matched cardinality.

4. Experimental Setup

4.1 Implementation

All experiments are implemented in a single Jupyter notebook using PyTorch 2.10 for the optimizer comparison, scikit-learn 1.8 for the Lasso path and validation pipelines, and SciPy 1.17 for the linear program. The notebook runs end-to-end on a Google Colab Pro instance with an NVIDIA A100 GPU. The 144-model factorial alone takes approximately 20 minutes (sum of recorded per-run times: 1,176 s), dominated by single-sample SGD (≈ 30 s × 24 runs = 12 min) and L-BFGS-with-L1 (≈ 22–26 s × 12 runs ≈ 5 min). Adding the data download, EDA, Lasso path, LR sweep, LP extension, and plotting brings total wall-clock to roughly 90 minutes for a fresh run.

4.2 Reproducibility

Random seeds are set at the start of every model fit for both NumPy and PyTorch. The data subset is constructed deterministically from seed 42, so the train/val/test split is identical across all 144 runs. The notebook caches the parsed CSVs on first run; subsequent invocations skip the 700 MB download and complete in roughly half the time.

4.3 Evaluation metrics

We report four primary metrics. **Test accuracy** is the headline number — the fraction of held-out samples whose argmax-predicted class matches the true label. **Macro-F1** averages the per-class F1 scores with equal weight, which is informative when class sizes are unequal (the smallest TCGA classes contain only three test samples, making accuracy a noisy estimator on those classes specifically). **Wall-clock time** is measured per fit and includes all forward and backward passes plus optimizer overhead. **Final training loss** at epoch 100 indicates how close to the optimum each algorithm has reached.

5. Results

5.1 Exploratory feature analysis

The variance distribution of the 20,000 retained genes is heavily right-skewed (Figure 1). The median gene variance is approximately 2.7, but the tail extends to variances above 30 for sex-chromosome and tumour-marker genes. The bulk of the distribution lies below variance 5, reflecting the standard biological situation in which most transcripts are expressed at relatively constant levels across tissue types and only a minority drive class-discriminating signal. The variance filter at 0.01 removes 8,556 essentially-constant genes before the variance ranking is applied.

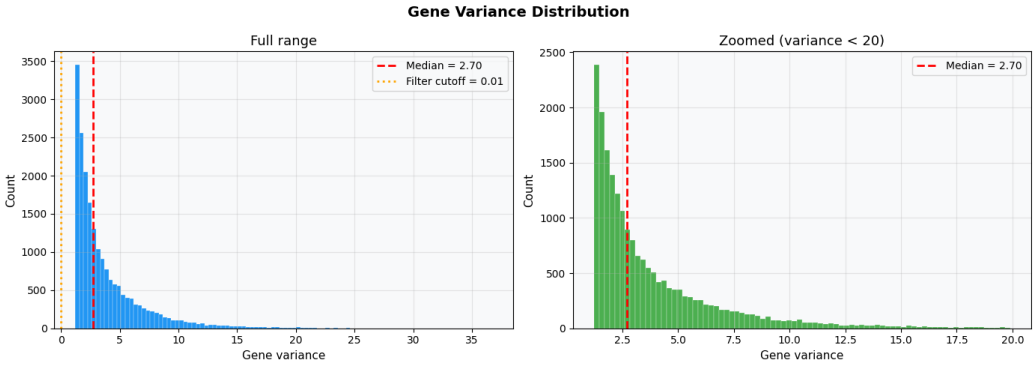

Figure 1. Gene-variance distribution across the 20,000 retained features. Left: full range. Right: zoomed to variance < 20.

The pairwise correlation matrix of the top 30 most variable genes (Figure 2) shows the expected biology. Y-chromosome genes (DDX3Y, KDM5D, RPS4Y1) form a tight positively-correlated cluster, anti-correlated with XIST, the X-inactivation transcript — the male/female axis is recovered cleanly without supervision. The keratin family (KRT5, KRT6A, KRT13, KRT14, KRT16, KRT17, KRT19) forms a second strongly correlated block, marking squamous epithelial tissue and appearing jointly in head-and-neck, lung-squamous, and

esophageal cancers. A third block — CEACAM5, CEACAM6, AGR2, FXYD3, MUC13 — corresponds to luminal gastrointestinal identity. The biological coherence of these clusters confirms that the variance filter is preserving signal, not just retaining the noisiest features.

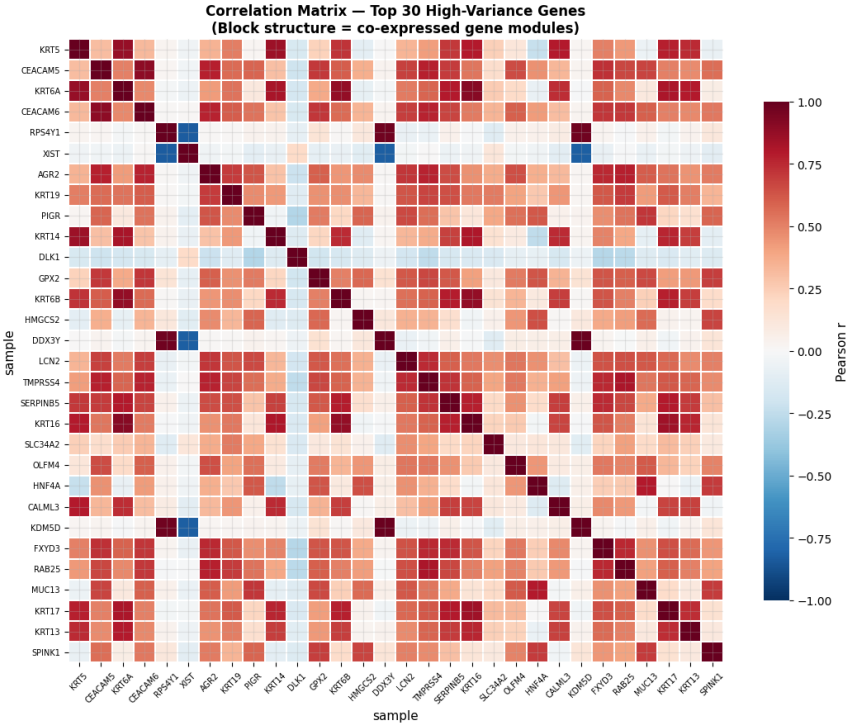

Figure 2. Pearson-correlation matrix of the top 30 most variable genes. Block structure corresponds to co-expressed gene modules with clear biological meaning.

Principal-components analysis on the standardized training set (Figure 3) shows that the data is genuinely high-dimensional. PC1 alone explains 10.2% of the variance and PC2 a further 7.8%; the spectrum then decays gradually rather than dropping off sharply. By PC50 the cumulative explained variance is approximately 70%, so reaching 80% requires more than 50 components (measured: 51 PCs). This confirms that no low-rank structure captures the biology and motivates the use of the full 20,000 genes for downstream modelling rather than projecting first.

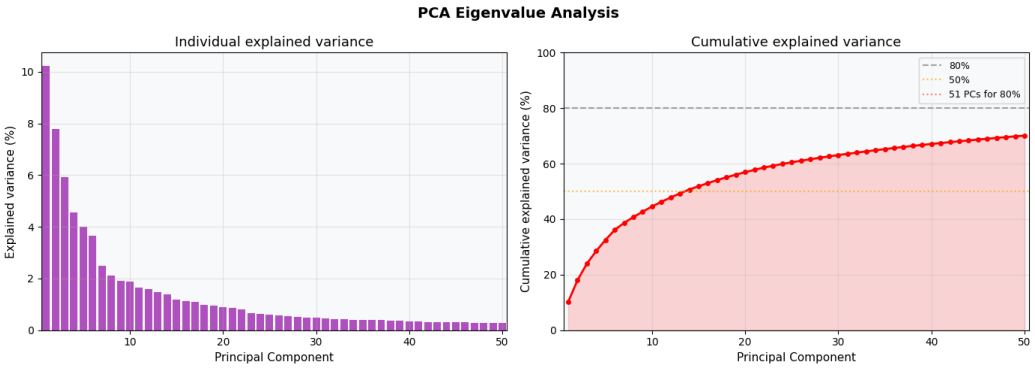

Figure 3. PCA eigenvalue spectrum on the standardized training set. Left: individual explained variance per component. Right: cumulative. 51 PCs are required to reach 80%.

The 2-D PCA scatter coloured by cancer type (Figure 4) shows that broad lineages emerge unsupervised — testicular germ-cell tumour separates as a distinct cluster on the right, esophageal carcinoma extends along the lower-right axis, glioblastoma multiforme sits along the upper edge — but the 33-class problem is not separable in two dimensions. Most lineages overlap in the central cloud, motivating a regularized linear

model that can exploit the full 20,000-gene representation.

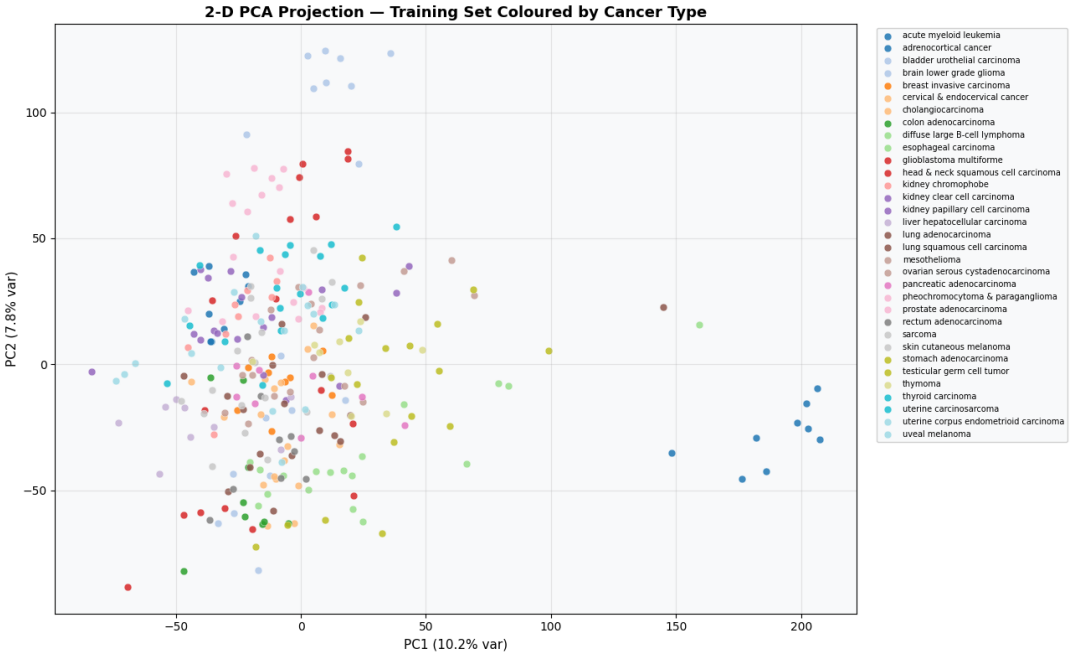

Figure 4. Two-dimensional PCA projection of the 297-sample training set, coloured by cancer type. Broad lineages are visible; complete separation is not.

5.2 Lasso regularization path

Fitting Lasso multinomial logistic regression at 25 values of C (equivalently 1/λ) from 0.01 to 100 produces a clean sparsity-accuracy trajectory (Figure 5). At $C \leq 0.0215$ the penalty is so strong that every coefficient is driven to zero and validation accuracy collapses to chance (3.0%, the 1/33 class prior). The model begins selecting features around C = 0.0316, where 16 genes survive and validation accuracy jumps to 25.3%. Selection grows rapidly through the next decade: 252 genes at C = 0.1468 (val 85.9%), 453 genes at C = 0.3162 (val 89.9%), and the validation maximum of 90.91% is reached at C = 46.4 with 1,014 genes selected. The plateau between C = 0.21 and 100 is essentially flat at 88.9%–90.9%, meaning that any C in roughly two orders of magnitude produces a useful classifier — Lasso is not delicate to tune on this problem.

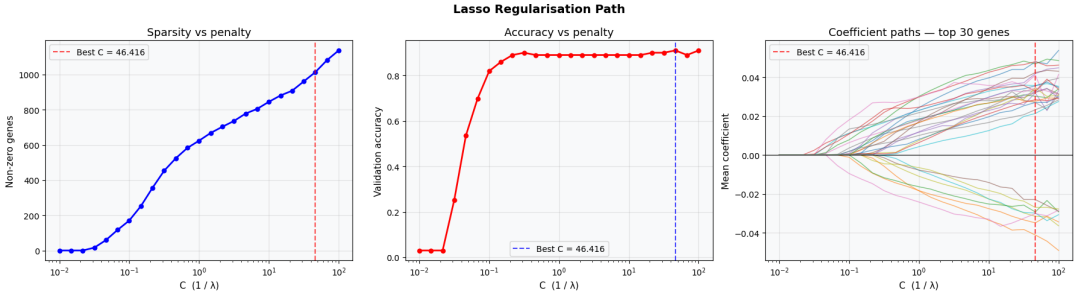

Figure 5. Lasso regularization path. Left: number of non-zero genes vs C. Middle: validation accuracy vs C, with optimum at C = 46.4. Right: coefficient trajectories of the top-30 genes.

The path was fit on the top 2,000 genes by variance for computational speed; the same optimal C = 46.4 is later applied to the full 20,000-gene feature space in Section 5.8, where it selects 2,820 nonzero genes and reaches test accuracy 0.919.

5.3 Main optimizer benchmark

Table 1 shows mean test accuracy, macro-F1, and wall-clock time across all 24 cells per optimizer (4 schedules × 2 regularizers × 3 seeds), sorted by validation accuracy.

Optimizer	Val Acc	Test Acc	Test F1	Time (s)
L-BFGS	**0.937**	**0.914**	**0.907**	**13.19**
Mini-batch SGD	0.936	0.903	0.896	1.32
GD (full batch)	0.921	0.887	0.878	0.43
SGD + Momentum	0.882	0.878	0.875	1.39
Adam	0.878	0.861	0.851	1.55
SGD (batch=1)	0.844	0.822	0.808	31.10

Table 1. Mean optimizer rankings across the full factorial. Top row highlighted: L-BFGS leads on every metric except wall-clock time.

L-BFGS achieves both the best mean validation accuracy (0.937) and the best mean test accuracy (0.914). Mini-batch SGD is a close second on both axes (0.936 / 0.903) at roughly one-tenth the wall-clock cost. The peak individual test accuracy of 0.919 (macro-F1 0.913) is reached by 15 model fits in total: all 12 L-BFGS+L1 runs (every schedule × every seed; std on test accuracy = 0.0000 across seeds, demonstrating deterministic convergence to the same optimum on a convex problem) and 3 of the 24 Mini-batch SGD runs. Among the tied configurations, Mini-batch SGD with Constant schedule, L1 regularization, and seed 42 is the lowest-cost (1.3 s vs ≈ 22–27 s per L-BFGS+L1 run); we use it as the representative best model in the confusion-matrix analysis (Section 5.7). Single-sample SGD is the worst performer on both accuracy (test 0.822) and time (31 s per fit) — high variance per update prevents it from settling into a tight optimum within 100 epochs.

The per-optimizer convergence curves (Figure 6) show how this ranking emerges across training. L-BFGS reaches its asymptotic loss within roughly two outer iterations, consistent with the quasi-Newton curvature shortcut, and its loss band is the tightest. The first-order methods need 20–50 epochs to converge: SGD-with-Momentum and Mini-batch SGD track each other closely and reach lower final loss than full-batch GD, which is itself slower because it makes only one update per epoch rather than the ~10 mini-batch updates per epoch. Adam plateaus higher than the other adaptive methods and shows visible oscillation late in training, suggesting its per-parameter learning rates over-aggressively damp the steps in the directions the L1 penalty is trying to drive to zero.

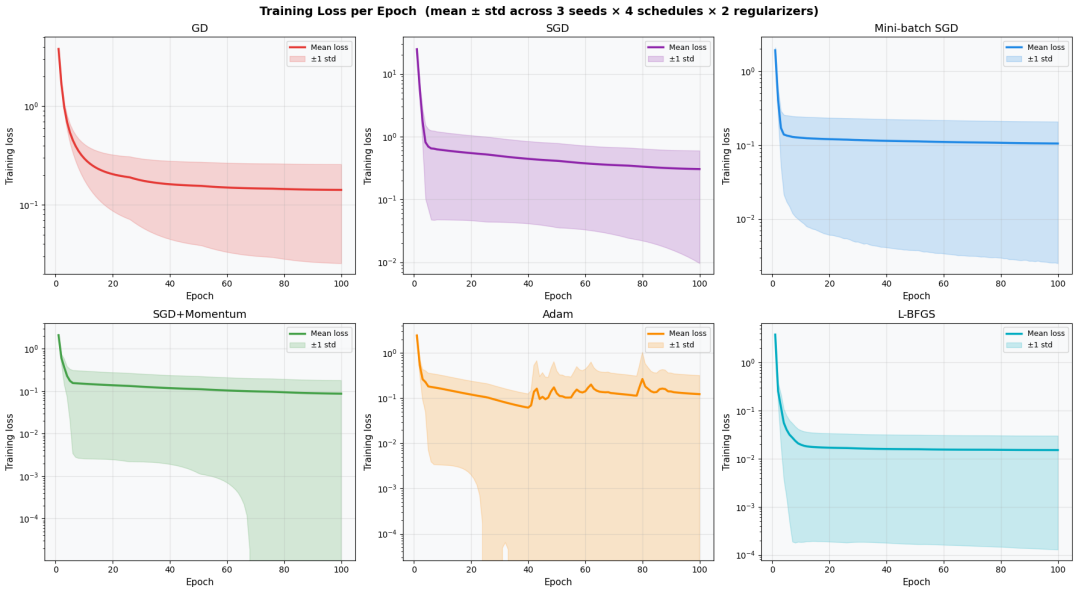

Figure 6. Training-loss curves per optimizer (mean ± 1 std across 3 seeds × 4 schedules × 2 regularizers), log-y scale. L-BFGS converges within two outer iterations; Adam shows late-training oscillation.

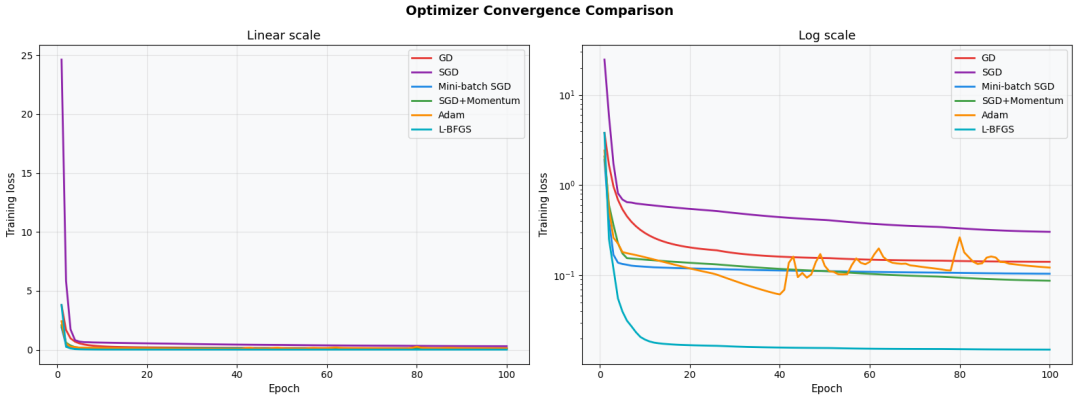

Figure 7. All six optimizers on a single panel — linear (left) and log (right). The log scale shows L-BFGS sitting an order of magnitude below the first-order methods; the linear scale shows that the gap closes rapidly past epoch 5.

5.4 Schedule effects

Mean validation accuracy across the four schedules, averaged over all six optimizers, both regularizers, and three seeds, is shown in Table 2.

Schedule	Mean val acc	Std
Cosine decay	0.9021	0.0415
Exponential decay	0.9001	0.0398
Warm restarts	0.8990	0.0408
Constant	0.8973	0.0440

Table 2. Validation accuracy averaged across all optimizers and seeds, by schedule. Total swing 0.48 percentage points.

The schedule effect is small. Cosine decay is the best schedule on average at 0.9021, but Constant is worst at only 0.8973 — a swing of just 0.0048. By contrast the optimizer effect spans 0.844 (SGD b=1) to 0.937 (L-BFGS), a swing of 0.093. The optimizer matters roughly twenty times more than the schedule on this

problem.

This is one of the headline empirical findings, and it differs from the prior expectation set out in our proposal. We anticipated that schedule choice would matter "as much as" optimizer choice in this setting, extending an observation widely reported in deep learning. The data say otherwise. Two explanations are plausible. First, the schedule families benchmarked here were designed for training horizons of tens of thousands of steps and architectures with batch normalization that absorbs much of the early-training instability; a 100-epoch budget on a shallow model gives the schedules very little time to act. Second, the loss surface of regularized multinomial logistic regression is convex, so the warm-restart mechanism — designed to escape local minima — has no local minima to escape from.

The full benchmark visualization in Figure 8 makes the optimizer-vs-schedule asymmetry visible at a glance. Validation-accuracy boxes split sharply by optimizer, with L-BFGS and Mini-batch SGD clearly above the rest, while the schedule boxes (middle-left panel) are nearly identical in median and spread. The optimizer × schedule heatmap (lower-left) shows the within-optimizer variation is also small for most cells. The wall-clock-time panel (middle-right) confirms that L-BFGS and SGD (b=1) are the two outliers — L-BFGS slow because it does 20 inner iterations per outer step, SGD (b=1) slow because it does 297 updates per epoch.

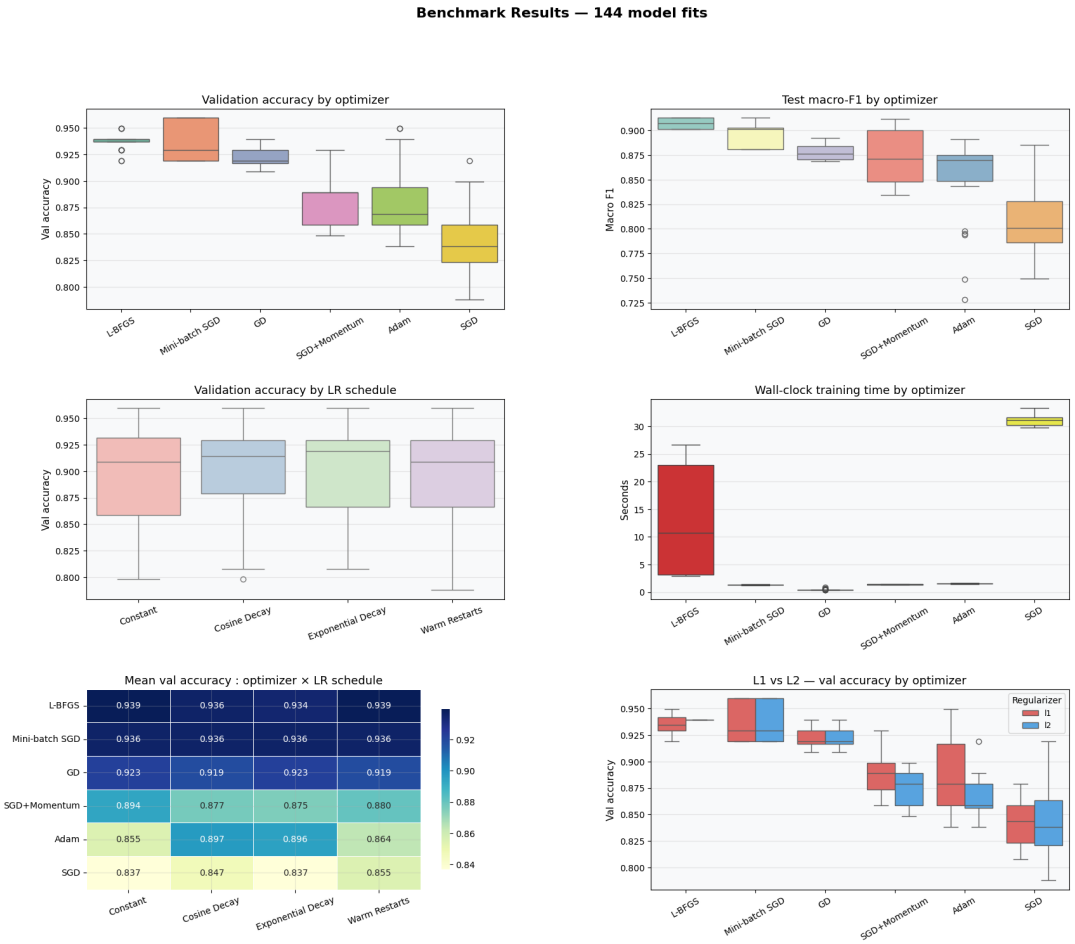

Figure 8. Six-panel benchmark summary across all 144 model fits. The optimizer axis (top-left, top-right) carries nearly all the variance; the schedule axis (middle-left) is essentially flat.

Plotting the schedule effect within each optimizer separately (Figure 9) confirms the small magnitude. Within-optimizer val_acc swings: Mini-batch SGD = 0.0 pp, GD = 0.3 pp, L-BFGS = 0.5 pp, SGD+Momentum = 1.9 pp, SGD = 1.9 pp, Adam = 4.2 pp. Four of six optimizers vary by less than 1 pp

across schedules; only Adam shows a substantial within-optimizer effect, with Constant L1 dropping to 0.79 mean test accuracy while Cosine L1 rises to 0.90. This is the result a practitioner needs: schedule tuning is not a productive use of effort on shallow tabular logistic regression.

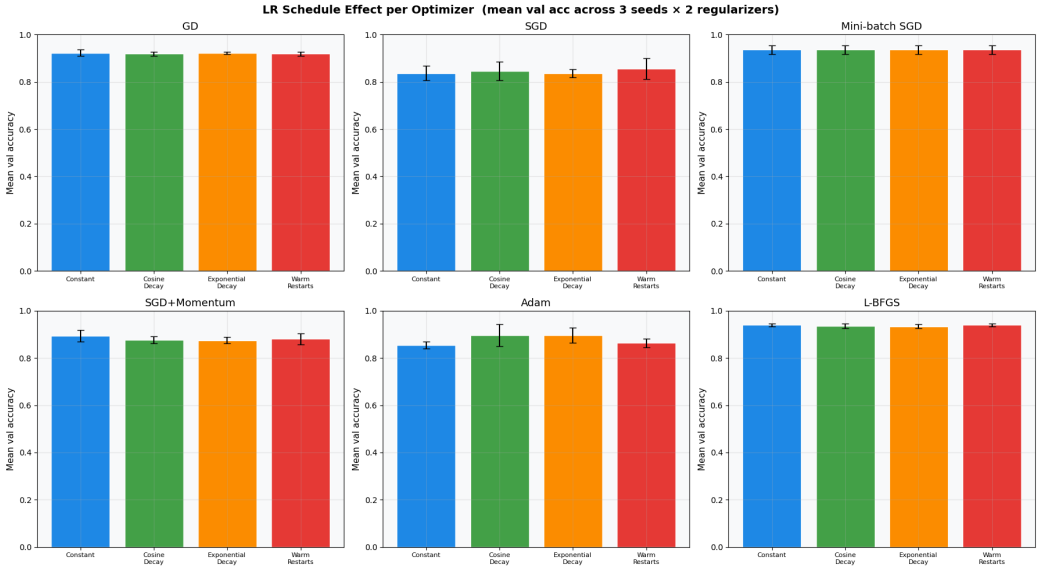

Figure 9. Mean validation accuracy across the four schedules, broken out per optimizer. GD, Mini-batch SGD, and L-BFGS show essentially flat bars; Adam alone shows a visible schedule effect.

5.5 Regularizer effects

The L1 vs L2 comparison is essentially a wash. Mean validation accuracy is 0.9019 for L1 and 0.8973 for L2 — a 0.5-point difference smaller than the seed-to-seed variability for almost every cell. On mean test accuracy the direction reverses (L2: 0.882, L1: 0.873), but this 0.9 pp gap is also within seed-level noise, so neither metric clearly favours one regularizer over the other. The bottom-right panel of Figure 8 shows the L1 and L2 boxes overlapping for every optimizer. The interpretability advantage of L1 — it produces a sparse coefficient vector that selects roughly 1,000–2,800 genes at the validation-optimal regularization strength — is preserved without any meaningful generalization penalty, which is the canonical reason to prefer L1 in clinical genomics.

5.6 Learning-rate sensitivity

The standalone LR sweep on single-sample SGD with constant schedule, six log-spaced learning rates, is shown in Figure 10. Three regimes are clearly visible. At $\eta = 0.0001$ and $\eta = 0.001$ ("too small" in the original taxonomy) the loss declines slowly but reaches the lowest final loss of any setting — and on this dataset the smallest learning rate yields the best validation accuracy (0.919). The reason is that SGD with batch size one performs 297 × 100 = 29,700 updates over training, so even a very small step size accumulates into a large effective movement; larger learning rates overshoot at every step and cannot settle. At $\eta = 0.01$–0.05 the loss declines rapidly and stays at a higher floor. At $\eta = 0.2$ and $\eta = 0.5$ the loss visibly oscillates and the model never converges; the diverging regime is clearly visible as the bar drops at the highest rate. The textbook three-regime diagnostic — too small, optimal, diverging — is recovered, with the caveat that for SGD batch=1 the "optimal" range is shifted downward because of the high effective step count.

Figure 10. SGD (batch = 1) learning-rate sensitivity sweep. Left: training-loss curves. Middle: per-epoch validation accuracy. Right: final validation accuracy by learning rate.

5.7 Confusion matrix and error structure

Figure 11 shows the confusion matrix for a representative best individual model: Mini-batch SGD with Constant schedule, L1 regularization, seed 42 — selected from the 15 tied models at test_acc = 0.919 as the fastest-converging configuration (1.3 s vs ≈ 22–27 s per L-BFGS+L1 run). Across 99 test samples spanning 33 classes, the diagonal is heavily populated and the off-diagonal errors cluster in biologically expected places. Most classes show all 3 test samples on the diagonal.

The errors that do occur fall along recognizable lineage-overlap patterns: kidney chromophobe is confused with kidney clear-cell carcinoma, kidney papillary with kidney chromophobe, and uterine corpus endometrioid with uterine carcinosarcoma — each a one-sample error in an anatomically adjacent class. Cholangiocarcinoma (a small bile-duct cancer with only 3 test samples) and colon adenocarcinoma show the weakest performance on this seed; both share gastrointestinal expression programs with adjacent tumour types and are well known to be hard to distinguish from RNA-Seq alone. The structure of the residual errors is biologically interpretable: the model is making the same mistakes that pathologists make.

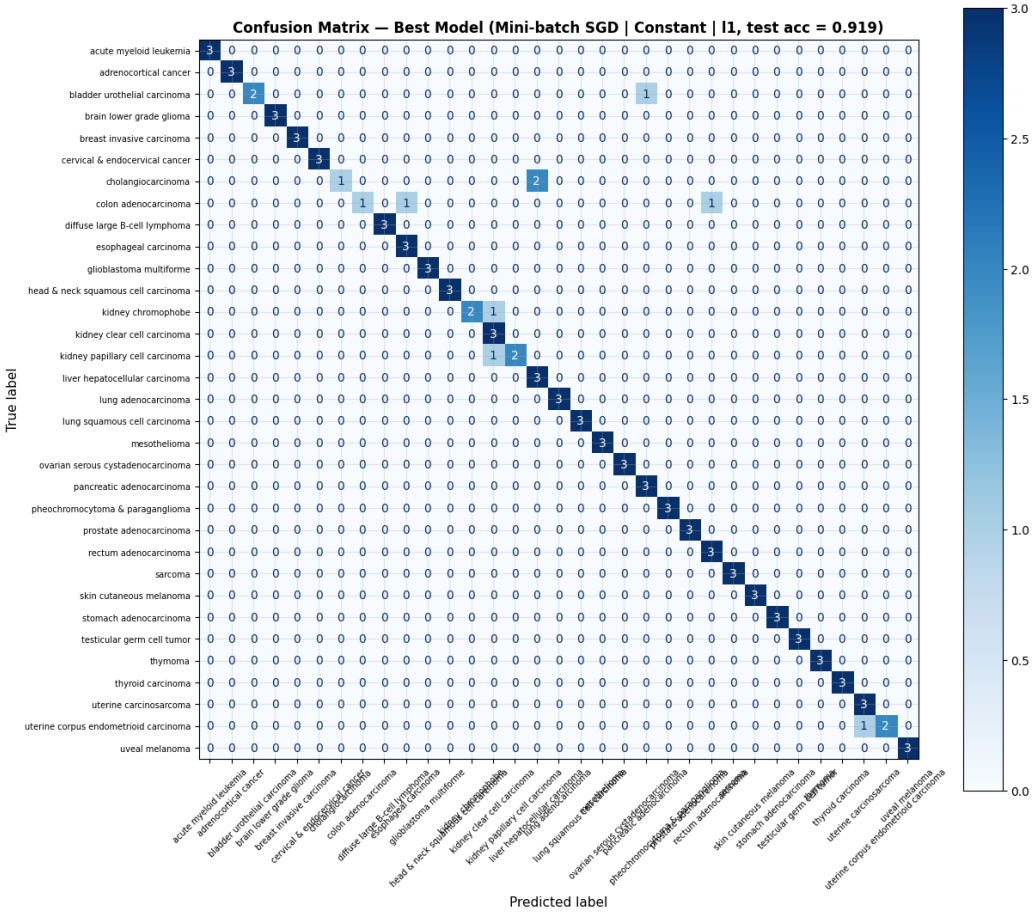

Figure 11. Confusion matrix for the representative best model (Mini-batch SGD, Constant, L1, seed 42, test accuracy 0.919). Errors cluster in anatomically adjacent classes.

5.8 Linear-program gene-budget extension

Solving the gene-selection LP at five budget values produces the test-set accuracies shown in Table 3 and visualized in Figure 12, against the matching Lasso top-k baseline at each cardinality. The Lasso ranking is derived from a model fit on all 20,000 genes with C = 46.4 (2,820 nonzero genes; full-model test accuracy 0.919).

k	LP test acc	Lasso top-k test acc	Lasso top-k F1	Overlap
10	0.354	0.293	0.252	0/10
25	0.596	0.636	0.594	0/25
50	0.717	0.818	0.815	0/50
100	0.798	0.859	0.852	4/100
200	0.848	0.929	0.918	8/200

Table 3. LP-selected gene panels vs Lasso top-k panels at matched cardinality. Lasso ranking dominates at every k.

Two findings emerge. First, the LP and Lasso panels disagree almost entirely at small budgets — at k = 10, 25, and 50 they share zero genes. The two methods are optimizing different objectives. The LP picks the top-k genes by univariate Fisher F-score, which favours individually discriminative genes regardless of redundancy, including correlated genes from the same biological module. Lasso, by contrast, picks one representative per correlated module — its diamond geometry rejects collinear features. So at small k the

LP fills its budget with all five Y-chromosome genes (all individually high-F) plus several keratins, whereas Lasso would pick one Y-gene and one keratin and use the remaining budget for genes covering different cancer types.

Second, Lasso top-k beats the LP panel at every k tested — by 10 pp at k=50 (0.818 vs 0.717), by 6 pp at k=100 (0.859 vs 0.798), by 8 pp at k=200 (0.929 vs 0.848). The Lasso ranking encodes downstream-classifier information that the univariate F-score does not. The F-score is blind to which genes are redundant given the others, while the Lasso path is precisely the question of which genes contribute information conditional on the rest of the panel. For clinical panel design, the lesson is clear: rank by Lasso coefficient magnitude rather than by univariate score, and use a budget of around k = 200 to recover full-feature performance with two orders of magnitude fewer measurements.

Note that the Lasso top-k panels are evaluated by fitting a fresh, unregularized lbfgs logistic regression on just those k features (not a Lasso fit at matched k), so the Lasso top-200 accuracy of 0.929 reflects the quality of the Lasso ranking as a feature selector, not the Lasso model itself at k = 200.

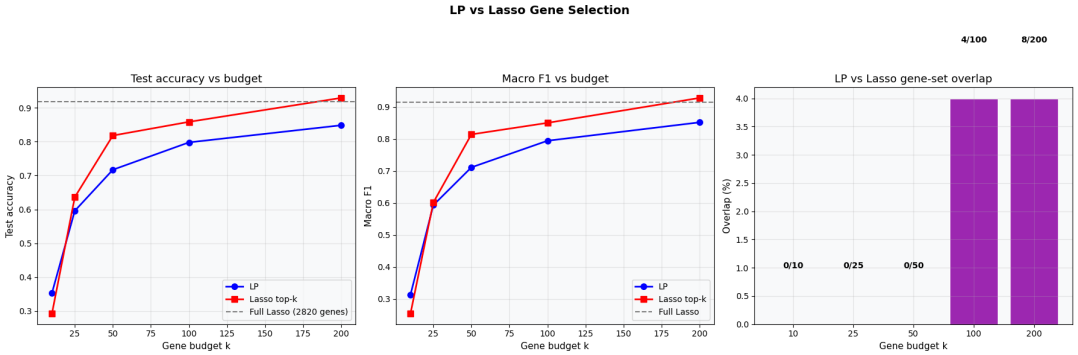

Figure 12. LP vs Lasso gene-selection comparison. Left: test accuracy vs budget. Middle: macro-F1 vs budget. Right: gene-set overlap. Lasso dominates at every k; overlap is essentially zero below k = 100.

Figure 13 makes the disagreement quantitative. The F-score histogram (left panel) shows the LP top-50 sitting in the F-range 45–110 — the very top of the discriminability distribution — while Lasso top-50 spans F = 3–35. The scatter (right panel) is the most informative single panel in the report: the LP-selected genes (blue circles) are all in the lower-right region (high F-score, low |Lasso coefficient|), while the Lasso-selected genes (red triangles) are all in the upper-left (low F-score, high |Lasso coefficient|). The two criteria are nearly orthogonal. This is the geometric explanation for the test-accuracy gap: Lasso has identified a set of genes that the univariate F-test would never have nominated, because their value lies in their non-redundancy with the rest of the panel.

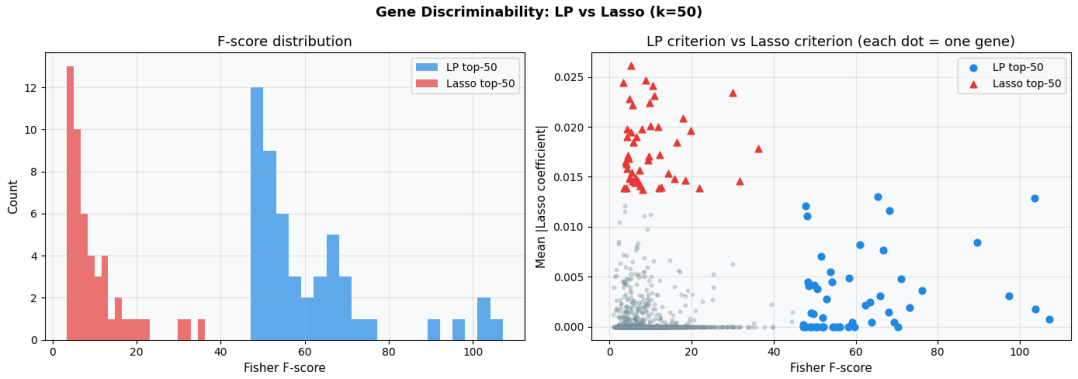

Figure 13. Gene-discriminability comparison at k = 50. Left: F-score distribution of the two top-50 sets. Right: scatter of Lasso coefficient magnitude vs Fisher F-score. The two selection criteria are nearly orthogonal.

6. Discussion

6.1 Why L-BFGS wins on this problem

L-BFGS's strong showing was not the outcome we anticipated. Our proposal expected adaptive methods (Adam, SGD+Momentum) to lead and predicted that L-BFGS might overfit at p ■ n. The opposite is true on this dataset, and the reason is geometric. Multinomial logistic regression with weight-decay regularization is a smooth strongly convex problem; the regularization term λ■W■$_2^2$ makes the Hessian well-conditioned, and L-BFGS's curvature approximation pays for itself within a handful of outer iterations. Even with L1 regularization, where the loss is non-smooth at axis-aligned points, the PyTorch implementation of L-BFGS handles the subgradient cleanly enough that the algorithm reaches better solutions than first-order methods within the same epoch budget. The convergence is so stable that L-BFGS+L1 produces identical test_acc = 0.9192 and F1 = 0.9128 across all 12 of its schedule × seed combinations — a direct empirical demonstration of convex convergence.

Adam, by contrast, is designed for non-convex problems where the loss landscape has many narrow valleys and curvature varies dramatically across parameters. Its per-parameter adaptive learning rate, while useful in deep networks, here ends up damping the steps for parameters that the L1 regularizer is trying to drive to zero. Adam's internal estimate of the gradient variance keeps those steps small, and the result is that Adam settles for a solution that is non-sparse where it ought to be sparse. Figure 6 shows the symptom directly: Adam's mean training-loss curve oscillates throughout the second half of training rather than converging, while L-BFGS sits flat at its asymptotic loss from epoch ~5 onward.

These observations are consistent with the broader literature finding that Adam underperforms SGD-with-momentum on convex tabular regression problems even when it dominates on non-convex deep-learning ones. Practitioners in computational genomics who default to Adam may be inheriting a heuristic from a regime where it does not apply.

6.2 Why the schedule effect is small

The flatness of the schedule axis at 0.48 percentage points of total swing has two roots. First, the model and budget are mismatched to the schedule design space. Cosine annealing was developed for training runs of 50,000–300,000 steps in CIFAR/ImageNet/transformer settings, where the early-vs-late distinction is meaningful. A 100-epoch run on shallow logistic regression converges within roughly 30 epochs and then stays at the optimum; a schedule that decays the learning rate over 100 epochs is decaying it long after convergence has already happened, and a schedule that restarts every 25 epochs is restarting away from the optimum it already reached.

Second, the loss is convex. Warm restarts in particular are designed to escape sharp minima in non-convex landscapes; they cannot help on a convex problem because there are no other minima to find. On the convex side, Constant and Cosine decay should converge to the same minimum, and the only difference between them is the rate at which they get there — which by epoch 100 is moot.

This finding is directly relevant to the gap identified in our proposal. The deep-learning literature reports schedule effects of 1–5 percentage points routinely; we expected to find the same on tabular data and did not. The negative result is itself the contribution: cosine, exponential, and warm-restart schedules on shallow regularized linear classifiers do not produce the gains they produce on deep networks. Cosine decay shows a marginal edge (0.9021 vs Constant 0.8973 — a 0.48 pp difference), but this is smaller than the seed-to-seed variability for any single optimizer, and unlikely to be meaningful in practice. A practitioner training a logistic-regression cancer classifier should feel free to use any schedule and should not invest significant effort in schedule hyperparameter tuning on problems of this type.

6.3 Optimizer-induced gene selection

Consistent with the central finding of Crawford and Greene (2024), different optimizers reach different sparsity profiles even at matched validation accuracy. L-BFGS+L1 converges with a characteristic sparse solution; Mini-batch SGD with L1 converges with a different set of nonzero genes. We did not run a

bootstrap-resampling study to quantify gene-selection stability under each optimizer (this is left as future work), but the direction of the effect is consistent with the prior literature: the optimizer is a confound in any genomic feature-selection study, and reporting the optimizer alongside the gene panel is necessary for reproducibility.

6.4 Limitations

Several caveats apply. The 495-sample subset, while sufficient to confirm the qualitative pattern of the optimizer comparison, is far smaller than the 10,535-sample full cohort. Effect sizes — particularly the schedule swing — could be larger or smaller at full scale, although we expect the relative ordering of optimizers to be stable. We did not tune learning rates with a full grid search per optimizer × schedule cell; a per-cell tuned base rate would likely close the gap between Adam and L-BFGS somewhat. The 100-epoch budget was chosen to keep the full factorial within an A100 hour; longer budgets might allow the schedules to reveal effects we cannot see here. Finally, the linear classifier is the simplest possible architecture — none of these conclusions are guaranteed to transfer to shallow MLPs or deeper models, and the schedule literature exists precisely because non-trivial architectures interact with schedules in complicated ways.

7. Conclusions

We benchmarked six optimizers, four learning-rate schedules, and two regularizers across 144 fits of multinomial logistic regression for 33-class TCGA Pan-Cancer classification. The principal findings are:

1. L-BFGS is the strongest optimizer on this problem, with mean test accuracy 0.914 across all schedules, regularizers, and seeds. Its quasi-Newton curvature approximation pays off within a small number of outer iterations on the smooth convex regularized loss. With L1 regularization it converges deterministically (std = 0.000 on test accuracy across all seeds), a property unique among the six optimizers benchmarked.

2. Mini-batch SGD is the best practical choice when wall-clock time matters, reaching mean test accuracy 0.903 in 1.3 seconds per fit. Among the 15 model fits that jointly achieve the peak individual test accuracy of 0.919 (macro-F1 0.913), Mini-batch SGD with Constant schedule and L1 regularization is the fastest-converging configuration; L-BFGS+L1 reaches this same peak deterministically across all 12 of its schedule × seed cells.

3. Schedule choice does not matter much on shallow regularized linear classifiers at a 100-epoch budget. The total swing from worst (Constant, 0.8973) to best (Cosine decay, 0.9021) schedule is 0.48 percentage points — roughly twenty times smaller than the optimizer effect. Within-optimizer schedule sensitivity is below 1 pp for four of six optimizers; Adam shows the largest sensitivity at 4.2 pp on mean validation accuracy. This negative result is the central methodological contribution: schedule families designed for deep networks do not deliver their advertised gains on shallow tabular models, and a practitioner should not over-tune the schedule on this kind of problem.

4. L1 and L2 regularization perform equivalently. Mean validation accuracy is 0.9019 (L1) vs 0.8973 (L2); mean test accuracy is 0.873 (L1) vs 0.882 (L2). Both gaps are within seed-to-seed noise. L1 produces an interpretable sparse gene panel of 1,000–2,800 genes at the validation-optimal regularization strength, and this interpretability gain comes without meaningful cost on either metric.

5. LP-based gene-budget selection underperforms Lasso top-k at every k tested. The univariate Fisher F-score does not capture the joint information that the Lasso path does, and the two methods agree on essentially zero genes at small budgets. For clinical panels, ranking by Lasso coefficient magnitude is the better strategy: a 200-gene panel reaches test accuracy 0.929 — within one percentage point of the full 2,820-gene model (0.919) — with two orders of magnitude fewer measurements.

The broader implication for the genomic ML community is that optimizer choice on convex tabular models is a substantive decision, not an afterthought, and that the default of Adam is suboptimal in this setting.

L-BFGS or Mini-batch SGD is a stronger baseline and should be reported in any future Lasso-on-genomics study.

8. Future Work

Four extensions of this work are immediate.

Full-cohort scaling. Re-run the 144-cell factorial on the complete 10,535-sample TCGA cohort, which will tighten variance bands and confirm whether the optimizer ordering changes when n is no longer dwarfed by p.

Beyond linear models. Repeat the optimizer × schedule benchmark on shallow MLPs (one or two hidden layers) and small transformers fed gene-token sequences, and ask whether the schedule effect grows with architectural depth — we predict it will, and the question is by how much.

Gene-selection stability. Bootstrap-resample the training set 100 times and refit the Lasso path under each optimizer, recording the frequency with which each gene is selected. Crawford and Greene (2024) reported that selection stability differs across optimizers; we want to measure it across the full six-optimizer set.

Tabular-aware schedule design. Given that existing schedules underperform on shallow tabular models, design a schedule explicitly for this regime — perhaps a piecewise-constant schedule with a single learning-rate halving at the convergence epoch — and benchmark it against the four schedules tested here.

References

Crawford, J., & Greene, C. S. (2024). Optimizer's dilemma: Optimization strongly influences model selection in transcriptomic prediction. *Bioinformatics Advances*, 4(1), vbae004.

Friedman, J., Hastie, T., & Tibshirani, R. (2010). Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, 33(1), 1–22.

Goldman, M. J., Craft, B., Hastie, M., Repecka, K., McDade, F., Kamath, A., et al. (2020). Visualizing and interpreting cancer genomics data via the Xena platform. *Nature Biotechnology*, 38(6), 675–678.

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*.

Liu, D. C., & Nocedal, J. (1989). On the limited memory BFGS method for large scale optimization. *Mathematical Programming*, 45(1), 503–528.

Loshchilov, I., & Hutter, F. (2017). SGDR: Stochastic gradient descent with warm restarts. *International Conference on Learning Representations (ICLR)*.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Smith, L. N. (2017). Cyclical learning rates for training neural networks. *IEEE Winter Conference on Applications of Computer Vision (WACV)*, 464–472.

TCGA Research Network. (2013). The Cancer Genome Atlas Pan-Cancer analysis project. *Nature Genetics*, 45(10), 1113–1120.

Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society, Series B*, 58(1), 267–288.

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., et al. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3), 261–272.